ER-PLATFORM-001

Interview prep — the full stack, the real problems hit while building it, and prepared answers.

Multi-tenant entity resolution SaaS for Indian school records. Solo-built: schema and RLS design, the full ML pipeline, Celery orchestration, the FastAPI layer, and a Next.js frontend. All facts below (test counts, versions, migration count) were re-verified against the running project on 2026-09-21, not recalled from memory.

1. The 30-second answer

> **SAY THIS** "It's a multi-tenant SaaS platform that deduplicates Indian school records — customers upload a CSV where the same school appears several times spelled differently, and the system decides which rows are the same place. The hard constraint was doing that without comparing every record to every other one, which is quadratic at 50,000 rows, and doing it with hard tenant isolation since multiple customers share one database. I built the schema and row-level security, the ML pipeline — a cold-start classifier, embeddings, vector blocking, probabilistic matching — the Celery orchestration, the FastAPI layer, and a Next.js review UI. 242 tests, all passing, run against real Postgres/Redis/Celery containers, not mocks."

2. Full technology inventory

Backend / data

TECHNOLOGY	VERSION	ROLE
Python	3.14.7	Runtime
FastAPI	0.141.1	API framework — async, request validation, OpenAPI
Pydantic v2	2.13	Request/response schemas, settings
SQLAlchemy 2.0	2.0.54	ORM — one async engine (API), one sync engine (Celery)
Alembic	1.20.0	Schema migrations, hand-written not autogenerated
psycopg 3	3.3.5	Postgres driver — same driver serves both engines
PostgreSQL	16	Primary store
pgvector	0.5.0 (client) / pg16 image	Vector column type + HNSW index
Redis	7.4	Celery broker, rate-limit token bucket
Celery	5.6.3	Distributed task queue, 9 queues
PyJWT	2.14.0	Token verification
httpx	0.28	Google Places client, test client
Docker Compose	—	Postgres + Redis for local dev

ML pipeline

TECHNOLOGY	VERSION	ROLE
XGBoost	3.4.1	Ingestion classifier (Valid / Junk / Missing_Context)
sentence-transformers	6.0.1	Embedding model wrapper (pulls in PyTorch)
paraphrase-multilingual-mpnet-base-v2	pinned by revision	The embedding model itself — multilingual for Devanagari/transliterated text
Splink	4.0.17	Probabilistic record linkage (Fellegi–Sunter)
DuckDB	via Splink	In-process SQL engine Splink runs its comparisons on
pandas	3.0.5	Feature frames, batch processing
NumPy	2.5.3	Vector math, union-find, weighted voting
scikit-learn	—	Splink dependency

Frontend

TECHNOLOGY	VERSION	ROLE
Next.js	15.5.25	App Router, server components, route handlers as an auth proxy
React	19.1.0	UI
TypeScript	^5	Type-checked client, hand-written API contract types
Tailwind CSS	v4	Styling — CSS-variable design tokens, dark mode via media query
jose	^6	Server-side JWT signing for the dev-token mock
Node.js	24.18.1	Runtime

Testing / tooling

TECHNOLOGY	ROLE
pytest / pytest-asyncio	242 tests: unit, API, and Docker-backed integration
pytest-timeout	120s hard cap per test — added after a real deadlock hung a test 10+ minutes
ruff / mypy	Lint + strict type checking, both clean across the codebase
ESLint / tsc	Frontend equivalent, both clean
Headless Chrome	Rendered this and the architecture doc to PDF

3. Architecture, in one diagram

```
CSV upload (202 Accepted, returns immediately)
  │
  ├─► INGEST     parse CSV in chunks, store rows verbatim as JSONB
  ├─► CLASSIFY   XGBoost: Valid / Junk / Missing_Context (weak-supervision cold start)
  ├─► EMBED      normalise text → 2×768-dim vectors per record (Valid rows only)
  ├─► BLOCK      pgvector HNSW nearest-neighbour search → candidate pairs → clusters
  ├─► MATCH      Splink Fellegi-Sunter scoring on every pair inside every cluster
  └─► GATE       route by the tenant's own thresholds:
        ≥ 0.950 → AUTO_MERGED   (no external call)
        ≥ 0.750 → PENDING_API   (Google Places validation, cached + rate-limited)
        < 0.750 → PENDING_REVIEW (a human decides)
```

Nine Celery queues, one per stage, independently scaled. Stages hand off *through Postgres*, not through Redis payloads — a 50k-row job produces roughly a million candidate pairs, and routing those through the broker would turn it into a data channel it was never sized for. Postgres Row-Level Security enforces tenant isolation independently of application code, so a forgotten `WHERE tenant_id = ?` is a bug caught at the database, not a breach.

4. War stories — real bugs, ready for "tell me about a hard problem"

Each of these is a genuine STAR answer: found under real load (mostly the Docker-backed integration suite or a live click-through, not a mock), with a root cause and a verified fix. Pick 2–3 depending on how technical the interviewer is — the RLS one and the Windows event-loop one are the strongest general-audience stories; the deadlock and the Splink calibration one are the strongest for a backend-focused interviewer.

War story 1 **SEVERITY: HIGH**

Row-Level Security caught a bug I didn't know I had — 5 of 6 write paths never set tenant_id

SITUATION

I'd built two independent tenant-isolation layers on purpose: the application always scopes queries by tenant, and Postgres RLS enforces the same rule at the database level as a backstop. The policy has a `WITH CHECK` clause, not just `USING` — it governs what a query is allowed to *write*, not just what it can see.

TASK

Run the first real end-to-end pipeline test against live Docker containers instead of mocks.

ACTION

The very first insert failed with "new row violates row-level security policy". I traced every write site in the codebase and found five of six — jobs, school_records, clusters, cluster_members, human_feedback — never set `tenant_id` explicitly. They relied on the session being tenant-bound, which governs what you can *see*, but the `WITH CHECK` clause requires the column to be set correctly on the row itself.

RESULT

Fixed all five sites, then wrote an integration test that inserts through the real API and confirms the row lands with the right tenant. This would have made every single upload fail in production, and no unit test would ever have caught it — you need a real Postgres evaluating a real policy.

War story 2 **SEVERITY: HIGH**

A self-inflicted deadlock that hung a worker for 10+ minutes, with no error

SITUATION

Every Celery stage wraps its work in a try/except that marks the job FAILED on error, so a failure is always diagnosable from Postgres alone, per the design requirement.

TASK

The error handler opens a *second* database session to write the failure — because the first session's transaction is about to roll back, and writing inside it would roll the failure message back too.

ACTION

What I missed: the failing transaction still held a row lock on that exact `jobs` row (from an earlier status update in the same request). The second session tried to update the same row and blocked on the lock. I caught it live with `pg_stat_activity` — one session `idle in transaction`, the other `waiting on Lock/transactionid` for over ten minutes. Postgres's own deadlock detector never fired, because the first session wasn't waiting on a lock at all — it was waiting on the client. No cycle, no detection, just a permanent hang.

RESULT

Fixed by rolling back the failing transaction *before* opening the second session. Added a hard `pytest-timeout` of 120 seconds to the whole test suite afterward, specifically so a regression like this fails loud instead of hanging CI silently.

War story 3 **SEVERITY: HIGH**

Windows made the entire async API non-functional, and no test caught it

SITUATION

242 tests passing, mypy and ruff clean, backend "done." First real HTTP request from the frontend to the API — every database-backed route returned 500.

TASK

Diagnose `psycopg.InterfaceError: Psycopg cannot use the 'ProactorEventLoop' to run in async mode`.

ACTION

Uvicorn hardcodes `ProactorEventLoop` on Windows, and psycopg's async mode refuses to run on it. My first fix — setting the asyncio event-loop *policy* in the app module — silently did nothing, because uvicorn passes an explicit `loop_factory` to `asyncio.run()`, which bypasses policies entirely. The working fix is a CLI flag: `--loop asyncio:SelectorEventLoop`, which is uvicorn's own escape hatch and happens to be the default behaviour on Linux/macOS anyway, so one command works everywhere.

RESULT

No test suite would have caught this without cost: the unit tests exercise the auth boundary only (never touching the DB), and the integration tests use Celery's *sync* engine. Only a real HTTP request through the async engine exposes it — which is exactly why I manually drove the app through the browser-equivalent flow before calling it done, rather than stopping at green tests.

War story 4 **SEVERITY: MEDIUM — CORRECTNESS**

A probabilistic matcher that ran clean and was silently useless

SITUATION

Splink's matcher needs three fitting steps: estimate `u` (agreement rate among non-matches), estimate `m` (agreement rate among matches) by Expectation-Maximisation, and estimate the prior probability that two random records match at all.

TASK

I implemented the first two and skipped the third as an apparent refinement.

ACTION

The model ran with no errors and ranked pairs in the right order — but genuine duplicates scored `0.043` and non-duplicates scored `0.000`. Since auto-merge triggers at 0.950, nothing would ever have auto-merged, ever, on any data. This is the dangerous kind of bug: it produces plausible-looking, silently wrong output rather than an error.

RESULT

Adding the missing prior-estimation step moved genuine duplicates to 0.87–0.91. Lesson I take into interviews: a model that runs without exceptions is not a model that is correct — test the actual output values, not just that the pipeline completes.

War story 5 **SEVERITY: MEDIUM — DATA INTEGRITY**

The multilingual model was chosen for Devanagari text, then a regex silently destroyed it

SITUATION

The embedding model was deliberately chosen to be multilingual, because Indian school names arrive transliterated inconsistently and sometimes not transliterated at all.

TASK

Write the text normaliser that runs before embedding — strip punctuation, expand abbreviations.

ACTION

My punctuation stripper kept "word characters" via `\w` and discarded the rest. Python's `\w` does not count Devanagari combining vowel marks as word characters, so `केंद्रीय विद्यालय` came back as a row of bare consonants — the exact text the multilingual model was chosen to handle, destroyed before it ever reached the model.

RESULT

Fixed by inverting the logic — *name* the punctuation to remove instead of naming the characters to keep. Caught by a test that asserted every Devanagari code point in the input survived normalisation, written specifically because "the model is multilingual" is a claim that needs its own test, not an assumption that falls out of the classifier tests passing.

War story 6 **SEVERITY: MEDIUM — FRONTEND**

A feature that looked finished and never worked once, because of how browsers navigate

SITUATION

The CSV export button was a plain `<a href="/v1/jobs/{id}/export">`, deliberately not a fetch call — the reasoning being that a real browser navigation streams a 50,000-row file without ever buffering it in JS memory, which is correct.

TASK

What that reasoning missed: a top-level browser navigation sends no `Authorization` header, and the backend route requires a bearer token.

ACTION

The button 401'd every single time — not intermittently. The steward's review page was replaced by the raw JSON body `{"detail":"Missing bearer token"}`, discarding their queue position. This was caught by an adversarial code-review pass I ran specifically to find issues like this — a reviewer traced the auth chain through four separate attempts to refute the claim and confirmed it every time.

RESULT

Fixed with a Next.js route handler that attaches the token server-side and pipes the response stream straight through — same memory property, the missing header fixed. Verified live: 401 before, 200 with a real streaming CSV after.

War story 7 **SEVERITY: LOW BUT MEMORABLE**

"It's up" was the wrong question — an HTTP 200 that meant nothing

SITUATION

I ran `next build` twice while `next dev` was live. Next.js shares the `.next` build directory between the two by default.

TASK

The production build overwrote the dev server's chunk manifests mid-flight. The user reported the page "not opening."

ACTION

Every server-side check passed: the port was listening, server-side rendering returned HTTP 200 with valid HTML, the backend health check was green. The actual failure was invisible to `curl`: the browser was told to fetch `main-app.js`, the React runtime itself, and got a 404. Blank page, forever, with every "is it up?" check lying.

RESULT

Fixed by giving builds a separate output directory (`.next-build`) so the collision is structurally impossible, then proved the fix by deliberately rerunning the exact sequence that broke it. The lesson: for any client-rendered app, checking that HTML came back is not checking that the app works — check that its JS assets actually returned 200 too.

5. Shorter bugs worth having ready

BUG	ONE-LINE CAUSE	ONE-LINE FIX
Classifier mislabeled all-empty rows	Missing-field heuristics outvoted junk heuristics on sheer count of blank fields	Gated the "missing context" rules on the row having a usable name first
XGBoost crashed on a 2-class cold start	`multi:softprob` needs sklearn to infer >2 classes; a tenant's first batch can lack one class entirely	Pick the objective at fit time based on classes actually present
pgvector + RLS "silent recall loss"	HNSW approximates *before* the tenant filter applies, so a crowded foreign tenant can starve genuine neighbours with zero error	Over-fetch (`LIMIT top_k × oversample`) + raise `hnsw.ef_search` ; proven with an integration test that places 300 foreign rows nearer the probe than any real neighbour
Keyset pagination broke on page 2 only	`(a, b) > (c, d)` on SQLAlchemy columns is a Python tuple comparison, not SQL — it evaluates `a == c` for truthiness and raises	Use SQLAlchemy's `tuple_()` for a real row-value comparison
`autoretry_for=(Exception,)` retried permanent errors	A missing file was retried 3 times, and each retry reported a different, misleading cause once the job was already FAILED	Replaced with an allowlist of genuinely transient exceptions (connection drops, timeouts)
A job's status machine could fail on its own success	A stray re-dispatch of a finished pipeline hit "job already resolved" and reported it as a hard failure, marking a healthy job FAILED	Distinguished "stale/already-done" from "illegal" as separate exception types with separate handling
Wrong DB credentials blocked everything at the start	A stray `.env` pointed at `postgres/postgres` instead of the actual provisioned role/database	Verified live against the container with `docker inspect` instead of trusting the file; deleted the stray file

6. Design decisions an interviewer will probe

Q. Why Postgres RLS instead of just filtering every query in application code?

A. Defense in depth. NFR was explicit: a single forgotten `WHERE tenant_id = ?` must not be a data breach. Application-level filtering is still the primary layer — RLS is the backstop that turns "someone missed a filter" into "the database rejected the write," which is exactly what happened in War Story 1.

Q. Why blocking before matching, instead of scoring every pair?

A. Quadratic cost. 50,000 rows is 1.25 billion pairs. Blocking uses cheap vector similarity to cut that down to a few thousand plausible pairs before the expensive probabilistic scorer ever runs.

Q. Why weak supervision for the classifier instead of collecting labels first?

A. Cold start — a brand-new tenant has zero labelled data on day one. Multiple cheap heuristic rules vote, and rows where they disagree are deliberately excluded from training rather than guessed at; those are exactly the rows that should go to a human first.

Q. Why is the confidence gate three bands instead of a single threshold?

A. Cost and trust calibration. High confidence auto-merges for free. Mid-confidence pays for one external validation call rather than either auto-merging on a shaky score or burning a human reviewer's time on something a cheap API call could settle. Low confidence always goes to a human — the system never guesses on a case it isn't sure about.

Q. Why hand-write Alembic migrations instead of autogenerating them?

A. Two things autogenerate doesn't round-trip reliably: HNSW indexes with operator classes and build parameters, and the RLS policies themselves, which are security-critical enough to want reviewed as literal SQL rather than trusted to a diff tool.

Q. Why does the frontend proxy through Next.js instead of calling FastAPI directly?

A. The backend has no CORS middleware, so a direct browser call from a different origin is blocked before FastAPI ever sees it. Proxying makes every call same-origin, and leaves adding real CORS as a deliberate production decision — an allow-list of real origins — rather than something loosened for local convenience.

7. Numbers worth having memorized

242	tests, all passing — unit, API-boundary, and Docker-backed integration
4	Alembic migrations, hand-written, RLS-guarded
9	Celery queues, independently scaled and rate-limited
2 × 768	embedding dimensions per record (name + address, independently, per FR-03)
0.950 / 0.750	per-tenant auto-merge / API-validation confidence thresholds
~1.25B → ~thousands	pairs before/after blocking, on a 50,000-row job
10+ min	the self-deadlock hung a worker before I diagnosed it live via `pg_stat_activity`
0.043 → 0.87–0.91	true-duplicate match score before/after fixing the Splink calibration bug

8. Questions to ask them back

TURN IT AROUND Good, genuine questions — not just "wrap-up" filler:

- "How do you handle multi-tenant data isolation — is it RLS, separate schemas, separate databases per tenant?"
- "What's your team's answer to 'the pipeline ran with no errors but the output was wrong' — do you have output-value assertions in CI, or mostly exception-based tests?"
- "How do you catch platform-specific bugs like the Windows event-loop one — do you run CI on more than one OS, or is dev/prod parity enough?"

9. If they ask what you'd do differently

HONEST ANSWER, PREPARED "Three things. First, I'd write the output-value assertions — not just 'does it run' — earlier for the ML stages; the Splink calibration bug should have been caught by a test that checks a known duplicate scores above some floor, not just by manual inspection. Second, I'd stand up the Docker integration suite before writing five write-paths that forgot `tenant_id`, not after — the sooner you're testing against a real database, the fewer of these compound. Third, the mid-confidence Google Places validation band is routing logic only; the actual external call behind it is unimplemented — that's the most honest gap in the project."
